

# TÊN ĐỀ TÀI: "CO-LAB WORKSPACE" – NỀN TẢNG QUẢN LÝ VÀ ĐẶT CHỖ KHÔNG GIAN LÀM VIỆC CHUNG THEO THỜI GIAN THỰC

## 1.1. Mô tả Bài toán & Tính Thực tế

Các nền tảng đặt phòng (booking) truyền thống thường chỉ cho phép đặt theo ngày hoặc theo gói cố định. Tuy nhiên, xu hướng làm việc linh hoạt (hybrid work) và nhu cầu của các freelancer đang tăng cao, dẫn đến một bài toán mới:

- **Người dùng (Freelancer, Nhóm dự án nhỏ):** "Tôi chỉ cần một bàn làm việc trong 3 giờ, một phòng họp trong 45 phút, hoặc một phòng thu âm trong 2 giờ. Tôi muốn thấy chính xác bàn/phòng nào còn trống *ngay bây giờ* và đặt ngay lập tức."
- **Người quản lý (Admin của Co-working space):** "Làm sao để tôi tối ưu hóa không gian của mình? Làm sao để quản lý các lượt đặt chỗ lẻ, check-in/check-out linh hoạt, và tránh tình trạng lãng phí tài nguyên?"

"Co-Lab Workspace" giải quyết vấn đề này bằng cách cung cấp một **bản đồ trực quan (visual map)** của không gian làm việc và cho phép **đặt chỗ theo thời gian thực**.

- Không phải là một trang e-commerce hay blog clone. Nó kết hợp giữa hệ thống quản lý (management system) và hệ thống đặt chỗ (booking system), kèm theo yếu tố **thời gian thực** và **trực quan hóa** (visual interface).
- Yêu cầu cho một đồ án tốt nghiệp với nhiều logic nghiệp vụ (booking, thanh toán, quản lý, phân quyền).

## 1.2. Áp dụng Bộ Công nghệ (Tech Stack)

- **Next.js (Frontend) + TypeScript:**
  - **SSR (Server-Side Rendering):** Dùng để render các trang công khai (trang chủ, giới thiệu không gian, trang chi tiết từng phòng). Điều này giúp **tăng cường SEO**, cho phép Google index các không gian làm việc của bạn.
  - **CSR (Client-Side Rendering):** Dùng cho **Dashboard (Bảng điều khiển)** người dùng và Admin, nơi các tương tác diễn ra liên tục mà không cần tải lại trang.
  - **Sơ đồ Tương tác (Visual Map):** Sử dụng các thư viện như react-leaflet (nếu có

nhiều chi nhánh) hoặc các thư viện vẽ SVG tương tác (như D3.js hoặc react-flow tùy biến) để vẽ sơ đồ không gian làm việc.

- **TypeScript:** Đảm bảo tính nhất quán của dữ liệu giữa các component (ví dụ: BookingProps, UserProps).
- **Express (Backend) + TypeScript:**
  - **Xây dựng API RESTful/GraphQL:** Cung cấp các endpoint cho Next.js gọi.
  - **Logic Nghiệp vụ Cốt lõi:** Xử lý các nghiệp vụ phức tạp như:
    - POST /api/bookings: Logic kiểm tra xung đột thời gian (conflict checking) – đây là phần quan trọng nhất.
    - GET /api/spaces/availability: Lấy trạng thái trống/bận của các bàn/phòng.
- **WebSocket (Tùy chọn nâng cao):** Tích hợp Socket.IO với Express để cập nhật trạng thái của bản đồ không gian làm việc (visual map) theo thời gian thực cho tất cả người dùng đang xem.
  - **TypeScript:** Định nghĩa các kiểu dữ liệu cho API request/response, đảm bảo an toàn kiểu (type-safe) khi giao tiếp với Next.js và MySQL.
- **MySQL (Database):**
  - **Lưu trữ Dữ liệu Quan hệ:** Hoàn hảo cho bài toán này. CSDL sẽ có các mối quan hệ phức tạp và rõ ràng:
    - workspaces (Các chi nhánh co-working)
    - spaces (Các tài nguyên: bàn làm việc, phòng họp, phòng thu âm... thuộc workspaces)
    - users (Người dùng, Admin, Chủ không gian)
    - bookings (Bảng ghi lại các lượt đặt chỗ, quan hệ N-1 với users và spaces, chứa start\_time, end\_time)
    - invoices (Hóa đơn thanh toán cho các bookings)
  - **Stored Procedures/Transactions:** Sử dụng MySQL Transactions để đảm bảo tính toàn vẹn dữ liệu khi đặt chỗ (ví dụ: khi kiểm tra phòng trống và tạo booking phải diễn ra như một hành động nguyên tử, không thể tách rời).

### 1.3. Các Phân hệ (Modules) chính

### 1. Phân hệ Người dùng (User):

- Xác thực (Đăng ký, Đăng nhập).
- Xem bản đồ/danh sách không gian.
- Lọc không gian (theo loại, sức chứa, tiện ích như máy chiếu, bảng trắng...).
- **Chức năng chính:** Chọn không gian, chọn khung giờ (ví dụ: 14:00 - 16:30), và đặt chỗ.
- Thanh toán (Tích hợp Stripe/VnPay).
- Quản lý lịch sử đặt chỗ.

### 2. Phân hệ Quản trị viên (Admin - Quản lý không gian):

- Dashboard quản lý tổng quan.
- **CRUD Không gian:** Tạo/Sửa/Xóa các chi nhánh, các phòng, các bàn làm việc.
- **Chức năng chính:** Thiết kế bản đồ trực quan (cho phép admin kéo/thả và định vị các bàn/phòng trên sơ đồ).
- Quản lý Đặt chỗ (Xác nhận, hủy, xem lịch).
- Quản lý Người dùng.
- Báo cáo doanh thu.

#### 1.4. Tính năng Nâng cao

- **Tích hợp WebSocket:** Khi Admin thay đổi trạng thái một bàn (ví dụ: "đang bảo trì") hoặc có người đặt chỗ, bản đồ trên máy của tất cả người dùng khác sẽ tự động cập nhật trạng thái của bàn đó sang màu đỏ (đã bận) mà không cần F5.
- **Hệ thống Tính giá Linh hoạt:** Tính tiền theo giờ (cho bàn làm việc) hoặc theo block 15 phút (cho phòng họp).
- **Check-in bằng Mã QR:** Khi đặt thành công, hệ thống tạo ra mã QR. Người dùng dùng mã này để quét tại quầy lễ tân (Admin dùng app/web để quét) để xác nhận check-in.

#### 1.5. Công nghệ Đề xuất

Bộ công nghệ này được chia làm 3 phần chính, tận dụng tối đa sức mạnh của TypeScript để đảm bảo an toàn dữ liệu từ Frontend đến Database.

### 1.5.1. Frontend (Client-Side & Server-Side)

**Nền tảng: Next.js + TypeScript** Lựa chọn này hoàn hảo vì bạn có thể xử lý cả trang public (SEO-friendly) và trang quản trị (dashboard) trong cùng một dự án.

– **Quản lý State (Trạng thái):**

- **Zustand** hoặc **React Context (+ useReducer)**: Thay vì dùng Redux phức tạp, Zustand là một thư viện quản lý state tối giản, hiệu quả, phù hợp để chia sẻ trạng thái người dùng (đã đăng nhập hay chưa), hoặc trạng thái của lượt đặt chỗ đang chọn.

– **Data Fetching (Gọi API):**

- **SWR (by Vercel)**: Đây là thư viện "ruột" cho Next.js. Nó giúp bạn gọi API từ Express một cách hiệu quả, tự động caching, và tự động "re-fetch" dữ liệu khi người dùng focus lại vào tab, đảm bảo dữ liệu luôn mới.

– **Form (Biểu mẫu):**

- **React Hook Form**: Thư viện tiêu chuẩn để quản lý các form phức tạp (đăng ký, đăng nhập, form đặt chỗ) với hiệu năng cao.
- **Zod**: Dùng để định nghĩa và xác thực (validate) schema dữ liệu. Bạn có thể dùng chung schema của Zod ở cả Frontend (validate form) và Backend (validate API body), đảm bảo tính nhất quán tuyệt đối.

– **UI/Component Library:**

- **Mantine UI** hoặc **Chakra UI**: Để xây dựng các Dashboard Admin và User, bạn cần rất nhiều component (Bảng, Modal, Form, Nút...). Sử dụng một component library sẽ giúp bạn tiết kiệm hàng tuần lễ. Cả hai đều có hỗ trợ tuyệt vời cho TypeScript và Next.js.

– **Bản đồ Trực quan (Visual Map):**

- **React Konva**: Đây là một thư viện React cho HTML5 Canvas. Nó rất mạnh mẽ cho việc xây dựng các giao diện 2D tương tác, cho phép bạn vẽ các Rect (phòng), Shape (bàn) và hỗ trợ sự kiện click (để chọn) và kéo/thả (cho Admin). Nó phù hợp hơn D3.js hay Leaflet cho bài toán bản đồ *trong nhà*.

– **Kết nối Real-time:**

- **Socket.IO Client:** Thư viện client của Socket.IO để lắng nghe các sự kiện (space-updated) từ server Express.

### 1.5.2. Backend (Server-Side)

**Nền tảng: Express + TypeScript** Nền tảng mạnh mẽ, linh hoạt để xây dựng API và xử lý logic nghiệp vụ.

#### – Giao tiếp CSDL (Database ORM):

- **Prisma:** Đây là lựa chọn ORM hiện đại và mạnh mẽ nhất cho TypeScript. Nó sẽ đọc CSDL MySQL của bạn và tự động tạo ra các type (kiểu dữ liệu) an toàn, giúp bạn truy vấn CSDL (ví dụ: `prisma.booking.findMany(...)`) với gợi ý code và kiểm tra lỗi ngay trong VS Code. Nó cũng xử lý Transactions rất tốt.

#### – Xác thực & Bảo mật:

- **Passport.js (với passport-jwt):** Một middleware tiêu chuẩn để xử lý xác thực. Bạn sẽ dùng nó để bảo vệ các API, đảm bảo chỉ người dùng đã đăng nhập mới có thể đặt chỗ.
- **Bcrypt.js:** Dùng để băm (hash) mật khẩu của người dùng trước khi lưu vào CSDL.
- **CORS:** Middleware cors để cho phép Next.js (chạy trên domain khác) có thể gọi API của Express.

#### – Xác thực API Body:

- **Zod (đồng bộ với Frontend):** Dùng Zod để đảm bảo dữ liệu (JSON body) mà Next.js gửi lên là hợp lệ trước khi Controller xử lý.

#### – Xử lý Real-time:

- **Socket.IO:** Tích hợp trực tiếp vào server Express để phát (emit) sự kiện mỗi khi có một lượt đặt chỗ mới hoặc Admin cập nhật trạng thái không gian.

### 1.5.3. Database (Cơ sở dữ liệu)

**Nền tảng: MySQL** Lựa chọn tốt cho dữ liệu có cấu trúc và quan hệ phức tạp như bài toán này.

#### – Công cụ Quản lý:

- **MySQL Workbench hoặc DBeaver:** Các công cụ trực quan để bạn thiết kế

ERD, xem dữ liệu và chạy các câu lệnh SQL thủ công khi cần.

– **Migration (Quản lý phiên bản CSDL):**

- **Prisma Migrate:** Nếu bạn dùng Prisma (đề xuất ở trên), nó đã tích hợp sẵn một hệ thống migration cực kỳ mạnh mẽ. Bạn chỉ cần sửa file schema.prisma, chạy một lệnh, và Prisma sẽ tự động tạo file script SQL để cập nhật CSDL.

**Tóm tắt Công nghệ Đề xuất (Tech Stack):**

| Phân hệ           | Công nghệ              | Mục đích                                   |
|-------------------|------------------------|--------------------------------------------|
| <b>Frontend</b>   | Next.js, TypeScript    | Nền tảng chính (SSR + CSR)                 |
|                   | Mantine UI / Chakra UI | Thư viện Component (Dashboard)             |
|                   | React Hook Form + Zod  | Quản lý và xác thực Form                   |
|                   | SWR (by Vercel)        | Data Fetching và Caching                   |
|                   | React Konva            | Xây dựng Bản đồ Trực quan (Kéo/Thả)        |
|                   | Socket.IO Client       | Lắng nghe sự kiện Real-time                |
|                   | NextAuth.js (Tùy chọn) | Đơn giản hóa luồng Đăng nhập/Session       |
| <b>Backend</b>    | Express, TypeScript    | Nền tảng xây dựng API                      |
|                   | Prisma                 | ORM (Thay thế cho việc viết SQL tay)       |
|                   | Passport.js (với JWT)  | Xác thực API                               |
|                   | Bcrypt.js              | Mã hóa Mật khẩu                            |
|                   | Socket.IO              | Phát sự kiện Real-time                     |
| <b>Database</b>   | MySQL                  | Lưu trữ dữ liệu quan hệ                    |
| <b>Thanh toán</b> | Stripe                 | Tích hợp cổng thanh toán quốc tế (để test) |